Introduction to Applied Cryptography

Thomas Peters – F.-X. Standaert

Slides 05



Key Agreement

How do A and B get their keys?

Key agreement:

- Pre-Internet: meet, run Gen, store k, leave
- Internet: no meeting, just public conversation

Can we create a shared secret from a public conversation?



Key Agreement (Intuition)

We would like to have a public box $[\cdot]_{pp}$

$$A \text{ picks } x \xrightarrow{[x]_{pp}} B \text{ picks } y$$

$$A \text{ picks } x \xleftarrow{[y]_{pp}} B \text{ picks } y$$

- A and B should be able to compute the same key k from either x and $[y]_{pp}$ or $[x]_{pp}$ and y, say $k := H([xy]_{pp})$
- Given only $[x]_{pp}$ and $[y]_{pp}$, computing $[xy]_{pp}$ must be hard

Challenges: Building boxes so that



- Adding in any box is easy
- Removing from a box or merging two boxes are hard

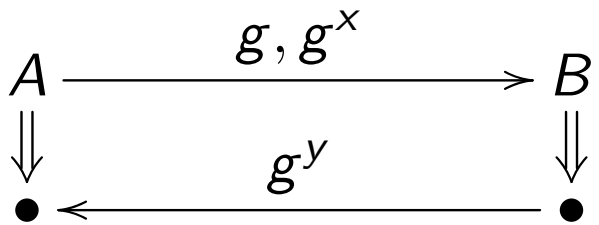
The Public Key Revolution



Merkle – Hellman – Diffie

The Diffie-Hellman Protocol (Outline)

The Diffie-Hellman protocol [DH76]



- A and B compute $k := g^{x \cdot y}$ as $(g^y)^x$ or $(g^x)^y$
- Computing $g^{x \cdot y}$ without x or y seems to require a logarithm extraction

Challenges:

- Can we make logarithm extraction difficult?
- Can we make sure that $g^{x \cdot y}$ does not become too big?

Asymmetric cryptography



Merkle – Hellman – Diffie

Groups

We can do DH with any set $\mathbb{G}$ equiped with a multiplication operation that "works well": we need $(g^x)^y = (g^y)^x$

We use:

Prime Order Cyclic Groups

That is:

- $\mathbb{G}$ is a group, i.e., a set equipped with operator "$\cdot$":
  - $\exists 1_{\mathbb{G}}$ (or just "1") s.t.: $\forall g \in \mathbb{G} : 1 \cdot g = g \cdot 1 = g$
  - $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
  - $\forall g \in \mathbb{G}, \exists h \in \mathbb{G} : g \cdot h = 1 = h \cdot g$ (h noted g^{-1})
- Cyclic: $\exists g \in \mathbb{G} : \mathbb{G} = \{g, g^2, g^3, \ldots, g^{|\mathbb{G}|}\}$
  Such a g is called a *generator*
- Prime Order: $|\mathbb{G}|$ is prime

Examples

- $\mathbb{Z}$ equipped with usual "$+$" is a group
- $\mathbb{Z}_n = \{0, \ldots, n-1\}$ with "add mod n" is a cyclic group
  $a + b \bmod n$ is the remainder of division of $a +_{\mathbb{Z}} b$ by n
- $\mathbb{Z}_p$ is a cyclic group of prime order if p is prime

Logarithm extraction

Can DH be secure in $\mathbb{Z}_p$?

Example: take $p = 17$, $g = 3$ and $x = 11$.

- Given $(g = 3, g^x = 16)$, can you compute x?
  (Remember: group operation is addition mod 17, but noted multiplicatively)

This is finding x s.t. $3x = 16 \bmod 17$

- x is $16 \cdot$ the **multiplicative inverse** of 3 mod 17
- Can we compute it efficiently (i.e., in PPT)?
  Does it exist for any p and g?

$\mathbb{Z}_p^*$

$\mathbb{Z}_p^* = \{1, \ldots, p-1\}$ with "mult mod p" is a group when p is prime

- *Ex:* $\mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$ and $3 \cdot 6 = 4 \in \mathbb{Z}_7^*$

Properties:

- $\exists 1_{\mathbb{G}}$ (or just "1") s.t.: $\forall g \in \mathbb{G} : 1 \cdot g = g \cdot 1 = g$
- $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- $\forall g \in \mathbb{G}, \exists h \in \mathbb{G} : g \cdot h = 1$ (h noted g^{-1})

Inversion in $\mathbb{Z}_p^*$

Let p be prime and $a \in \mathbb{Z}_p^*$

- Is there $b : a \cdot b = 1$?
- Can we compute this b?

We need $b, k \in \mathbb{Z} : ab + kp = 1 \in \mathbb{Z}$

Multiplicative inverse mod N

More general statement:

Proposition: a is invertible mod N iff $\gcd(a, N) = 1$

$\Rightarrow$ Suppose b is the inverse of a mod N

- $ab = 1 \pmod{N} \Rightarrow \exists c : ab - 1 = cN$
- Then $ab - cN = 1$.
  Suppose $d|a$ and $d|N$. Then $d|1 \Rightarrow d = 1$.
  $\Rightarrow \gcd(a, N) = 1$



Multiplicative inverse mod N

Proposition: a is invertible mod N iff $\gcd(a, N) = 1$

- $\Leftarrow$ Suppose $\gcd(a, N) = 1$
  - We show $\exists X, Y : Xa = YN + 1$ (then $X = a^{-1}$)
  - Let $d^* = Ua + VN$ be the smallest integer of $\mathcal{S} := \{\hat{U}a + \hat{V}N : \hat{U}, \hat{V} \in \mathbb{Z}\} \cap \mathbb{N}^{\geq 1}$
  - Fix any $c := U'a + V'N$, $c \in \mathcal{S}$
    Express $c = qd^* + r$ with $0 \leq r < d^*$.
  - We have $r = U'a + V'N - q(Ua + VN) \Rightarrow r = 0$
  - So, $d^* | c$, and
    $d^* | a \in \mathcal{S}$ and $d^* | N \in \mathcal{S}$ (take $(U', V') = (0, 1)$ or $(1, 0)$)
    $\Rightarrow d^* = 1$



Multiplicative inverse mod N

Corollary: if a is invertible mod N then it has one only inverse in $[0, N)$

- Suppose $a \cdot b = a \cdot b' = 1 \pmod{N}$

  $\Rightarrow a \cdot (b - b') = 0 \pmod{N}$

  $\Rightarrow b - b' = 0 \pmod{N}$, since $\gcd(a, N) = 1$

Corollary: if p is prime, then every element of $\mathbb{Z}_p^*$ has an inverse

Can we compute the inverse?

Let $\gcd(a, p) = 1$, we need $b, k \in \mathbb{Z} : ab + kp = 1 \in \mathbb{Z}$

Trick of the **Extended Euclidian Algorithm**:

$$\text{If } a < p, \text{ write } p = qa + r \text{ with } 0 \leq r < q$$
$$\text{So, solve equation } ab' + kr = 1$$

Observations:

- We have $b' = b + kq$, so solution of 2nd equation gives solution to 1st equation
- 2nd equation looks for inverse of r mod a, and $a < p$ so recursion can work
- If $a \leq p/2$ then modulus lost one bit
  If $a > p/2$ then $r < p/2$, and modulus will lose one bit next
  So efficient recursion of depth at most $2|p|$

Extended Euclidian Algorithm

Extended Euclidian algorithm eEucl (twisted):

in: a, b, with $a \geq b > 0$
out: (X, Y) where $Xa + Yb = 1$

if $b = 1$ **then return** $(0, 1)$
else $(q, r) := (\lfloor a/b \rfloor, [a \bmod b])$
(X', Y'):=eEucl(b, r)
return $(Y', X' - Y'q)$

Observe, if $(q, r) := (\lfloor a/b \rfloor, [a \bmod b])$

- if $X'b + Y'r = 1$ then
  $X'b + Y'(a - bq) = 1$
  $Y'a + (X' - Y'q)b = 1$

Extended Euclidian Algorithm

Compute $(X, Y) = \text{eEucl}(a, b)$
if $b = 1$ **then return** $(0, 1)$
else $(q, r) := (\lfloor a/b \rfloor, [a \bmod b])$
(X', Y'):=eEucl(b, r)
return $(Y', X' - Y'q)$

Example:

			q	r
Call	57	53	1	4
Call	53	4	13	1
Call	4	1		

- Last recursive call outputs $(0, 1)$
- $1st$ unwinding outputs $(1, -13[= 0 - 1 \cdot 13])$
- $2nd$ unwinding outputs $(-13, 14[= 1 - (-13) \cdot 1])$
- Correctness: $1 = -13 \cdot 57 + 14 \cdot 53 = -741 + 742$
  So $53^{-1} = 14 \in \mathbb{Z}_{57}^*$

Logarithm extraction

Can DH be secure in $\mathbb{Z}_p$?

Example: take $p = 17$, $g = 3$ and $x = 11$.

- Given $(g = 3, g^x = 16)$, can you compute x?
  (Remember: group operation is addition mod 17, but noted multiplicatively)

This is finding x s.t. $3x = 16 \bmod 17$

And computing multiplicative inverses can be done in PPT
So, no, DH protocol is not secure in $\mathbb{Z}_p$

$\mathbb{Z}_p^*$

$\mathbb{Z}_p^* = \{1, \ldots, n-1\}$ with "mult mod p" is a group

- *Ex:* $\mathbb{Z}_{17}^* = \{1, 2, \ldots, 16\}$ and $3 \cdot 7 = 4 \in \mathbb{Z}_{17}^*$

What about computing logarithms in $\mathbb{Z}_p^*$?

Example: take $p = 17$, $g = 3$ and $x = 11$.

- Given $(g = 3, g^x = 7)$, can you compute x?
  (Remember: group operation is multiplication mod 17)
- No obvious way apart from exhaustive search
  But how long would it be?
- Depends of **order** of g, i.e., smallest $i : g^i = 1$
  Powers of
  $\langle 3 \rangle = \{3, 9, 10, 13, 5, 15, 11, 16, 14, 8, 7, 4, 12, 2, 6, 1\} = \mathbb{Z}_{17}^*$
  But powers of $\langle 16 \rangle = \{16, 1\}$

Fermat's little theorem

Let $\mathbb{G}$ be a commutative group with $m := |\mathbb{G}|$
Then, $\forall g \in \mathbb{G}, g^m = 1$

Proof:

- Let $g_1, \ldots, g_m$ be the elements of $\mathbb{G}$
- Observe $g_1 \cdots g_m = (gg_1) \cdots (gg_m)$
  (all terms of the right-hand product must be distinct)
- Multiply both sides by $(g_1 \cdots g_m)^{-1}$

Corollaries:

- $g^i = g^{[i \bmod m]}$
- $\forall g \in \mathbb{G}$, if $\mathrm{ord}(g) = i$, then $i|m$
  (Otherwise, $g^{[m \bmod i]} = 1$)

How to define $\mathbb{G}$ from $\mathbb{Z}_p^*$

1. Find primes p and q such that $p = 2kq + 1$ for some k.
2. Pick a random g_0 over $\mathbb{Z}_p^*$ and compute $g = g_0^{2k} \bmod p$
3. If $g \neq 1$, set $\mathbb{G} = \langle g \rangle$, else, go back to 2.

What if $p = 2q_1 \cdot q_\ell + 1$ with small primes $q_1, \ldots, q_\ell$?

$\mathbb{Z}_p^*$

Can we find g s.t. $\mathrm{ord}(g) = p - 1$?

Theorem (Gauss, 1801):

$$\exists g_0 \in \mathbb{Z}_p^* \text{ s.t. } \mathrm{ord}(g_0) = p - 1 \text{ for every prime } p$$

And these *primitive roots* are quite common:

$$\Pr[g_0 \text{ is a primitive root}] = \frac{1}{\log \log p}$$

when g_0 and p are picked uniformly at random



The Discrete Logarithm Problem

Removing from the box $[\cdot]_{pp}$ should be hard

Consider experiment $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $h \xleftarrow{R} \mathbb{G}$
3. Set $x \leftarrow \mathcal{A}(\mathbb{G}, q, g, h)$
4. $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow g^x = h$

The *discrete logarithm problem* is hard relative to $\mathcal{G}$ if
$\forall$ PPT $\mathcal{A}$, there is a negligible ϵ s.t.:

$$\Pr[\mathsf{DLog}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \epsilon(n)$$

The Discrete Logarithm Problem in $\mathbb{Z}_p^*$

1. How to choose p?
2. How to choose g?



Generic Algorithms for the DL

Given an instance $(\mathbb{G}, q, g, h)$, how can we find x?

In a *generic* way, that is, assuming that the only available operation is the multiplication in $\mathbb{G}$

1. Try all x until $g^x = h$
   Takes up to q attempts, so $|q| = 128$ looks good
2. Baby-step, giant-step algorithm, with $s = \sqrt{q}$:
   (i) Compute g^s (naive solution takes $\mathcal{O}(\sqrt{q})$ steps)
   (ii) Compute $S = \{g^s, g^{2s}, \ldots, g^{(s^2)}\}$ (takes $\mathcal{O}(\sqrt{q})$ steps)
   (iii) Search $i \in \{0, \ldots, q\}$ s.t., if $g^i h = g^{js} \in S$
   (takes $\mathcal{O}(\sqrt{q})$ steps)
   (iv) Return $x = js - i$

   Takes $\mathcal{O}(\sqrt{q})$ work, so $|q| = 256$ looks good

No asymptotically better solution is known.

Non Generic Algorithms for the DL

Given an instance $(\mathbb{Z}_p^*, q, g, h)$, how can we find x?
Can we do better than generic algorithms?
E.g., exploit the fact that addition $\bmod p$ works too?

Yes:

- Index calculus: $\approx \mathcal{O}(2^{|p|^{1/2}(\log|p|)^{1/2}})$ [Kraitchik, 1922]
- NFS: $\approx \mathcal{O}(2^{|p|^{1/3}(\log|p|)^{2/3}})$ [Lenstra, Lenstra, 1993]

So, today:

- $|p| = 3072$
- $|q| = 256$: we actually work in a subgroup

How to find a big prime p?

How to select a large prime p?

- Select random integer, test primality, retry if failure

Can this work?

- Is primality testing easier than factoring?
  Can we test primality using a PPT algorithm?
- How many failures should we expect?
  What is the prime numbers' density?



Primality testing

Is primality testing easier than factoring?

> *"In August 2002 one of the most ancient computational problems was finally solved."* [1]

[Agrawal-Kayal-Saxena 2002]:



Prime is in **P**!

[1] 2006 Gödel Prize citation

Density of Primes

Prime number theorem [de la Vallée-Poussin 1896], [Hadamard 1896]

$$\Pr[N \text{ is prime}] \approx \frac{constant}{|N|}$$



$|N|^2$ attempts makes failure probability negligible. . .

Exponentiation in Groups

How to compute g^x?

Is it easy to put x in the box $[\cdot]_{pp}$?

- $g \cdot g \cdot g \cdots g$ (i.e., $x - 1$ multiplications)
  Problem: complexity is exponential in $|x|$!
- Better idea: $g^8 = ((g^2)^2)^2)$
  So: g^{2^i} can be computed with $i - 1$ multiplications
- General version: **Square and Multiply** algorithm

$$g^x = \begin{cases} (g^{\frac{x}{2}})^2 & x \text{ is even} \\ g \cdot (g^{\frac{x-1}{2}})^2 & x \text{ is odd} \end{cases}$$

  Requires only $|x|$ iterations...



What about other groups?

Is there any cryptographically useful group besides $\mathbb{Z}_p^*$?

Yes!
Groups defined on the
points of elliptic curves

[Koblitz 1985],
[Miller 1985]



Neal Koblitz

Elliptic Curve Cryptography (ECC)

Groups defined on the points of elliptic curves. . .

Is this useful?

- Harder to break
  Best DL extraction algorithms are generic
- We can use shorter keys
  $\mathbb{Z}_p^*$ 3072 bits often compared to ECC 256 bits
- Faster algorithms
  Specially useful in constrained environments



Elliptic Curve Cryptography (ECC)

Groups defined on the points of elliptic curves. . .

Why don't we always use ECC?

- Do not put all your eggs in the same basket. . .
- $>$ 100 patents held by US companies
  - efficient multiplication on EC
  - efficient point representation on EC
  - . . .

  Considerably slowed down adoption!

Elliptic Curves

Groups defined on the points of elliptic curves

Elliptic curves are defined by the equation:

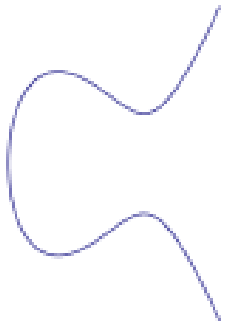
$$y^2 = x^3 + Ax + B \mod p$$

$E(\mathbb{Z}_p) := \{(x, y) \in \mathbb{Z}_p \times \mathbb{Z}_p \text{ on the curve } \} \cup \mathcal{O}$

Examples:



$y^2 = x^3 - 2x$



$y^2 = x^3 - 2x + 2$

Elliptic Curves

Example: $y^2 = x^3 - 2x + 2 \mod 7$

Only 4 elements of $\mathbb{Z}_7$ have square roots

- $0 = 0^2$; $1 = 1^2 = 6^2$; $2 = 3^2 = 4^2$; $4 = 2^2 = 5^2$

Consider $f(x) := x^3 - 2x + 2 \mod 7$

- $f(0) = 2 \Rightarrow (0, 3)$ and $(0, 4) \in E(\mathbb{Z}_7)$
- $f(1) = 1 \Rightarrow (1, 1)$ and $(1, 6) \in E(\mathbb{Z}_7)$
- $f(2) = 6$, but 6 has no square roots mod 7
- $f(3) = 2 \Rightarrow (3, 3)$ and $(3, 4) \in E(\mathbb{Z}_7)$
- $f(4) = 2 \Rightarrow (4, 3)$ and $(4, 4) \in E(\mathbb{Z}_7)$
- $f(5) = 5$, but 5 has no square roots mod 7
- $f(6) = 3$, but 3 has no square roots mod 7

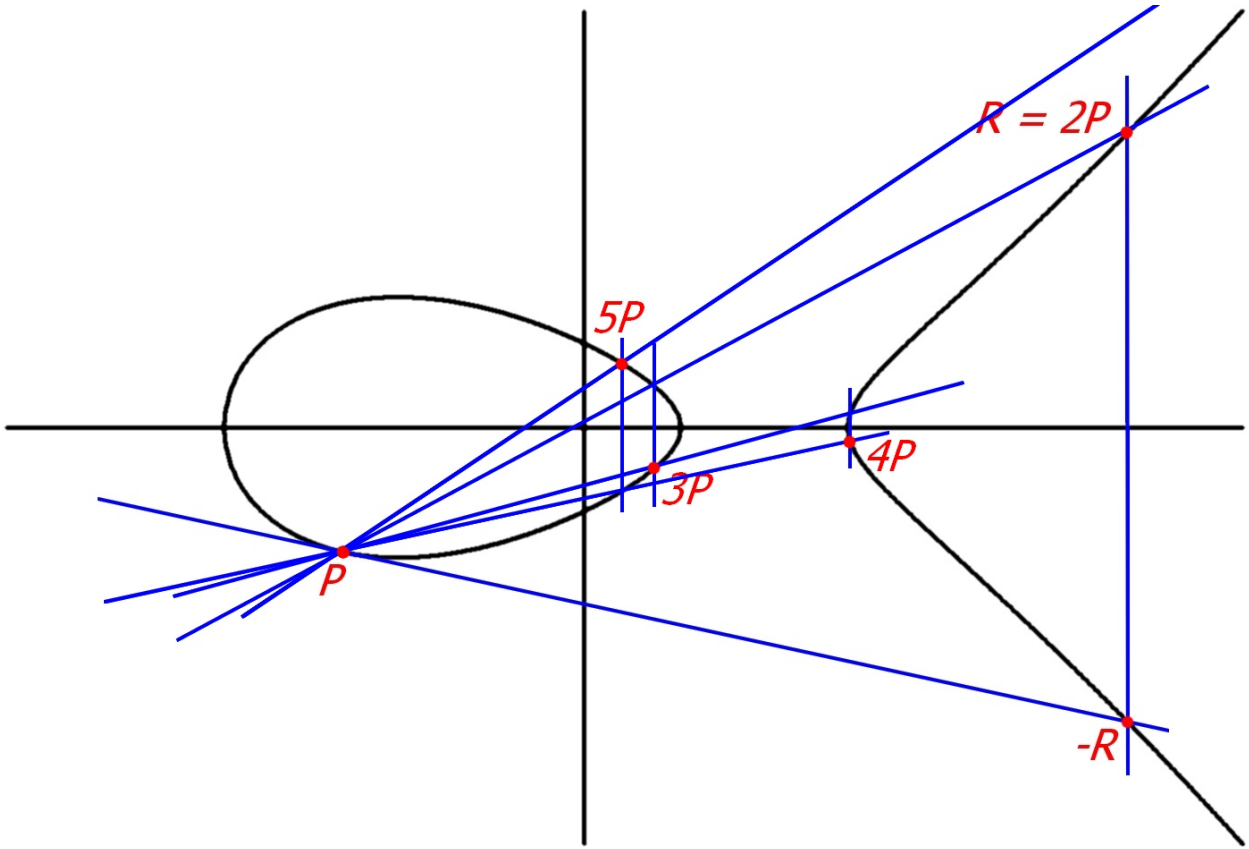
$\Rightarrow |E(\mathbb{Z}_7)| = 9$ (including $\mathcal{O}$)



Elliptic Curve Cryptography

We have a group operator and can consider DL on EC:

- ECDLP: Given points P and aP, find a



Pics by P. Bulens.

Elliptic Curve Cryptography

Schemes based on generic group operations can be used in the EC setting...

- How to choose a curve?
- What is the order of the group obtained?
- How to translate a message into a point?
- How to represent this point efficiently?
- How to compute sums, multiplications efficiently?

Many standards exist[2]

[2] See, e.g., `https://safecurves.cr.yp.to/`



Diffie-Hellman Key Agreement

Usage in TLS: [3]

	Key exchange/agreement and authentication					
Algorithm	**SSL 2.0**	**SSL 3.0**	**TLS 1.0**	**TLS 1.1**	**TLS 1.2**	**TLS 1.3**
RSA	Yes	Yes	Yes	Yes	Yes	No
DH-RSA	No	Yes	Yes	Yes	Yes	No
DHE-RSA (forward secrecy)	No	Yes	Yes	Yes	Yes	Yes
ECDH-RSA	No	No	Yes	Yes	Yes	No
ECDHE-RSA (forward secrecy)	No	No	Yes	Yes	Yes	Yes
DH-DSS	No	Yes	Yes	Yes	Yes	No
DHE-DSS (forward secrecy)	No	Yes	Yes	Yes	Yes	No[43]
ECDH-ECDSA	No	No	Yes	Yes	Yes	No
ECDHE-ECDSA (forward secrecy)	No	No	Yes	Yes	Yes	Yes

[3] Credit: Wikipedia

DH Key Exchange Security

We need more than the DL assumption!

It should be hard to merge two boxes $[x]_{pp}$ and $[y]_{pp}$!

- It may be possible to compute g^{xy} without solving the DL (We do not know how, but we cannot exclude it)
- We need g^{xy} to be unpredictable, else on can derive k

Actually g^{xy} should look random in $\mathbb{G}$, not just hard to compute



Computational Diffie-Hellman Problem

Consider experiment $\mathrm{CDH}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $(x, y) \xleftarrow{R} \mathbb{Z}_q^2$
3. Set $h \leftarrow \mathcal{A}(\mathbb{G}, q, g, g^x, g^y)$
4. $\mathrm{CDH}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow h = g^{x \cdot y}$



Computational Diffie-Hellman Problem

The *computational Diffie-Hellman problem* is hard relative to $\mathcal{G}$ if $\forall$ PPT $\mathcal{A}$, there is a negl. ϵ s.t.:

$$\Pr[\mathsf{CDH}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \epsilon(n)$$

Observe:

- If $\mathsf{DLog}_{\mathcal{A},\mathcal{G}}$ is easy then $\mathsf{CDH}_{\mathcal{A},\mathcal{G}}$ is easy
- The converse is not necessarily true!
- The CDH assumption states that the CDH problem is hard

Decisional Diffie-Hellman Problem

We can even want that $[xy]_{pp}$ looks random

Consider experiment $\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n)$

1. Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$ where g generates the group $\mathbb{G}$ of order q, with $|q| = n$
2. Choose $(x, y, z) \xleftarrow{R} \mathbb{Z}_q^3$
3. Flip a coin $b \xleftarrow{R} \{0,1\}$, and set $h_1 := g^{x \cdot y}$ and $h_0 := g^z$
4. Set $b' \leftarrow \mathcal{A}\big(\mathbb{G}, q, g, (g^x, g^y, h_b)\big)$
5. $\mathrm{DDH}_{\mathcal{A},\mathcal{G}}(n) = 1 \Leftrightarrow b = b'$

Observe:

- No need to *compute* g^{xy}: just *decide* whether you see g^{xy} or g^z

Decisional Diffie-Hellman Problem

The *decisional Diffie-Hellman problem* is hard relative to $\mathcal{G}$ if $\forall$ PPT $\mathcal{A}$, there is a negl. $\epsilon(n)$ s.t.:

$$\Pr[\mathsf{DDH}_{\mathcal{A},\mathcal{G}}(n) = 1] \leq \frac{1}{2} + \epsilon(n),$$

Observe:

- If $\mathsf{CDH}_{\mathcal{A},\mathcal{G}}$ is easy then $\mathsf{DDH}_{\mathcal{A},\mathcal{G}}$ is easy
- The converse is believed to be false
- The DDH assumption states that the DDH problem is hard

